

e-Commerce consumer behaviour

1. Importing required libraries
2. Loading the DataSet
3. Number of Rows and columns
4. DataFrame information
5. Checking missing values
6. Checking duplicate values
7. Number of unique values
8. Handle missing values in days_since_prior_order column
9. Converting days_since_prior_order into int data type
10. groupby on order_dow(Day of the week) column
11. Number of purchases by day
12. Time of day, when the order was made
13. Keep the transactions separate like, morning, afternoon, evening, night
14. The time when the order was made by day and time of day
15. The number of orders that users have made
16. How many days have passed since the pre-order
17. Top 15 most popular products from users
18. How many products users add to the cart
19. How many products were added to the cart
20. Reorderedes
21. Top 10 reorders

In []:

1. Importing required libraries

In [1]:
`import numpy as np
import pandas as pd

import seaborn as sns`

2. Loading the DataSet

In [2]: `df = pd.read_csv('ECommerce_consumer_behaviour.csv')`

In [3]: `df.head()`

Out[3]:

	order_id	user_id	order_number	order_dow	order_hour_of_day	days_since_prior_order	product_id	add_to_cart_order	reordered	department_id	department	product_name
0	2425083	49125	1	2	18	NaN	17	1	0	13	pantry	baking ingredients
1	2425083	49125	1	2	18	NaN	91	2	0	16	dairy eggs	soy lactosefree
2	2425083	49125	1	2	18	NaN	36	3	0	16	dairy eggs	butter
3	2425083	49125	1	2	18	NaN	83	4	0	4	produce	fresh vegetables
4	2425083	49125	1	2	18	NaN	83	5	0	4	produce	fresh vegetables

3. Number of rows and columns

In [4]: `df.shape`

Out[4]: (2019501, 12)

4. DataFrame information

In [5]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2019501 entries, 0 to 2019500  
Data columns (total 12 columns):  
#   Column              Dtype  
---  ---  
0   order_id            int64  
1   user_id             int64  
2   order_number        int64  
3   order_dow           int64  
4   order_hour_of_day   int64  
5   days_since_prior_order float64  
6   product_id          int64  
7   add_to_cart_order    int64  
8   reordered           int64  
9   department_id       int64  
10  department          object  
11  product_name        object  
dtypes: float64(1), int64(9), object(2)  
memory usage: 184.9+ MB
```

5. Checking missing values

In [6]: `df.isna().sum()`

Out[6]:

order_id	0
user_id	0
order_number	0
order_dow	0
order_hour_of_day	0
days_since_prior_order	124342
product_id	0
add_to_cart_order	0
reordered	0
department_id	0
department	0
product_name	0
dtype: int64	

6. Checking duplicate values

In [7]: `df.duplicated().sum()`

Out[7]: 0

7. Number of unique values

In [8]: `df.nunique()`


```
Out[8]: order_id      200000
       user_id      105273
       order_number  100
       order_dow     7
       order_hour_of_day  24
       days_since_prior_order  31
       product_id     134
       add_to_cart_order  137
       reordered      2
       department_id   21
       department     21
       product_name    134
       dtype: int64
```

8. Handle missing values in days_since_prior_order column

```
In [9]: df['days_since_prior_order']

Out[9]: 0      NaN
       1      NaN
       2      NaN
       3      NaN
       4      NaN
       ...
       2019496    5.0
       2019497    3.0
       2019498    3.0
       2019499    3.0
       2019500    3.0
       Name: days_since_prior_order, Length: 2019501, dtype: float64
```

```
In [10]: df['days_since_prior_order'] = df['days_since_prior_order'].fillna('0')
```

```
In [11]: df['days_since_prior_order']

Out[11]: 0      0
       1      0
       2      0
       3      0
       4      0
       ...
       2019496    5.0
       2019497    3.0
       2019498    3.0
       2019499    3.0
       2019500    3.0
       Name: days_since_prior_order, Length: 2019501, dtype: object
```

9. Converting days_since_prior_order into int data type

```
In [12]: df['days_since_prior_order'] = df['days_since_prior_order'].astype(int)
```

```
In [13]: df['days_since_prior_order']

Out[13]: 0      0
       1      0
       2      0
       3      0
       4      0
       ..
       2019496    5
       2019497    3
       2019498    3
       2019499    3
       2019500    3
       Name: days_since_prior_order, Length: 2019501, dtype: int32
```

In []:

```
In [14]: df.groupby('order_dow')['user_id'].agg(['count'])
```

```
Out[14]:      count
order_dow
0      391831
1      349236
2      261912
3      238730
4      234884
5      262157
6      280751
```

In []:

10. groupby on order_dow(Day of the week) column

```
In [15]: df.groupby('order_dow')['user_id'].agg(['count'])
```

```
Out[15]:      count
order_dow
0      391831
1      349236
2      261912
3      238730
4      234884
5      262157
6      280751
```

```
In [16]: df.groupby('order_dow')['user_id'].agg(['count']).sort_values(by = 'count')
```

```
Out[16]:      count
order_dow
4      234884
3      238730
2      261912
5      262157
6      280751
1      349236
0      391831
```

```
In [17]: df.groupby('order_dow')['user_id'].agg(['count']).sort_values(by = 'count', ascending = False)
```

```
Out[17]:      count
order_dow
0      391831
1      349236
6      280751
5      262157
2      261912
3      238730
4      234884
```

```
In [18]: result = df.groupby('order_dow')['user_id'].agg(['count']).sort_values(by = 'count', ascending = False)
```

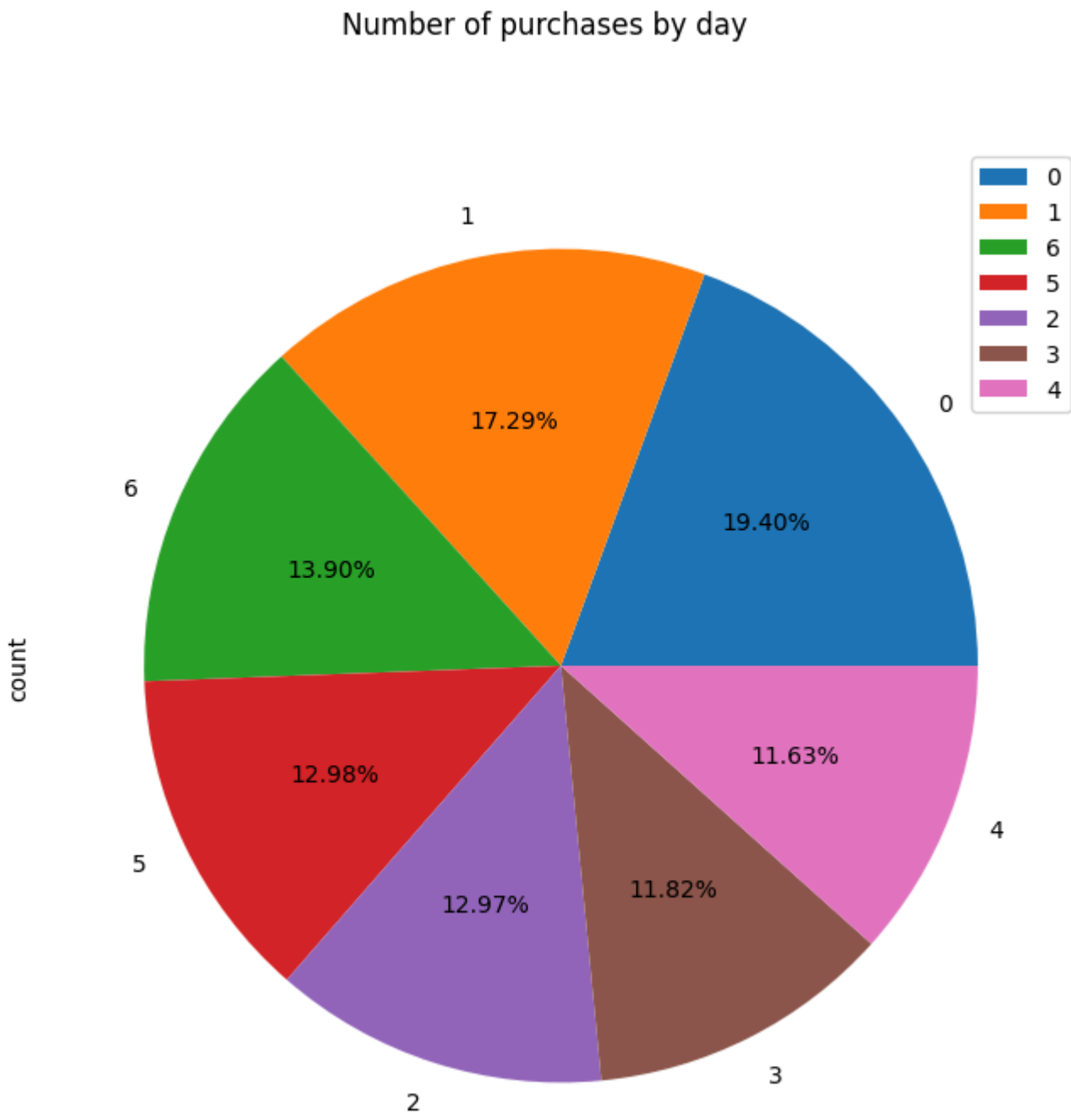
```
In [19]: result
```

```
Out[19]:      count
order_dow
0      391831
1      349236
6      280751
5      262157
2      261912
3      238730
4      234884
```

11. Number of purchases by day


```
In [20]: result.plot(
        kind = 'pie',
        autopct = '%1.2f%%',
        subplots = True,
        title = 'Number of purchases by day',
        figsize = (8, 8)
    )
```

Out[20]: array([<Axes: ylabel='count'>], dtype=object)



12. Time of day, when the order was made

```
In [21]: result1 = df.groupby('order_hour_of_day', as_index=False).agg({'user_id': 'count'})
```

```
In [22]: result1
```

Out[22]:

order_hour_of_day	user_id
0	13481
1	7283
2	4210
3	2994
4	3269
5	5732
6	18293
7	54143
8	106754
9	150248
10	173306
11	170291
12	163511
13	166376
14	167831
15	167157
16	158247
17	129383
18	102416
19	78516
20	62110
21	48857
22	40762
23	24331

```
In [23]: result1.sort_values(by='user_id', ascending=False)
```

Out[23]:

order_hour_of_day	user_id
10	173306
11	170291
14	167831
15	167157
13	166376
12	163511
16	158247
9	150248
17	129383
8	106754
18	102416
19	78516
20	62110
7	54143
21	48857
22	40762
23	24331
6	18293
0	13481
1	7283
5	5732
2	4210
4	3269
3	2994

13. Keep the transactions separate like, morning, afternoon, evening, night

```
In [24]: def order_time(x):
        if x in [6,7,8,9,10,11,12]:
            return 'Morning'
        if x in [13,14,15,16,17]:
            return 'Afternoon'
        if x in [18,19,20,21,22]:
            return 'Evening'
        if x in [23,24,0,1,2,3,4,5]:
            return 'Night'
        return x
```

```
In [25]: df['order_time_list']=df['order_hour_of_day'].apply(order_time)
```

```
In [26]: df.head()
```


Out[26]:

	order_id	user_id	order_number	order_dow	order_hour_of_day	days_since_prior_order	product_id	add_to_cart_order	reordered	department_id	department	product_name	order_time_list
0	2425083	49125	1	2	18	0	17	1	0	13	pantry	baking ingredients	Evening
1	2425083	49125	1	2	18	0	91	2	0	16	dairy eggs	soy lactosefree	Evening
2	2425083	49125	1	2	18	0	36	3	0	16	dairy eggs	butter	Evening
3	2425083	49125	1	2	18	0	83	4	0	4	produce	fresh vegetables	Evening
4	2425083	49125	1	2	18	0	83	5	0	4	produce	fresh vegetables	Evening

In [27]:

```
result2 = df.groupby('order_time_list')['user_id'].agg(['count'])
```

In [28]:

```
result2
```

Out[28]:

	count
order_time_list	
Afternoon	788994
Evening	332661
Morning	836546
Night	61300

In [29]:

```
result2 = result2.sort_values(by = 'count',ascending = False)
```

In [30]:

```
result2
```

Out[30]:

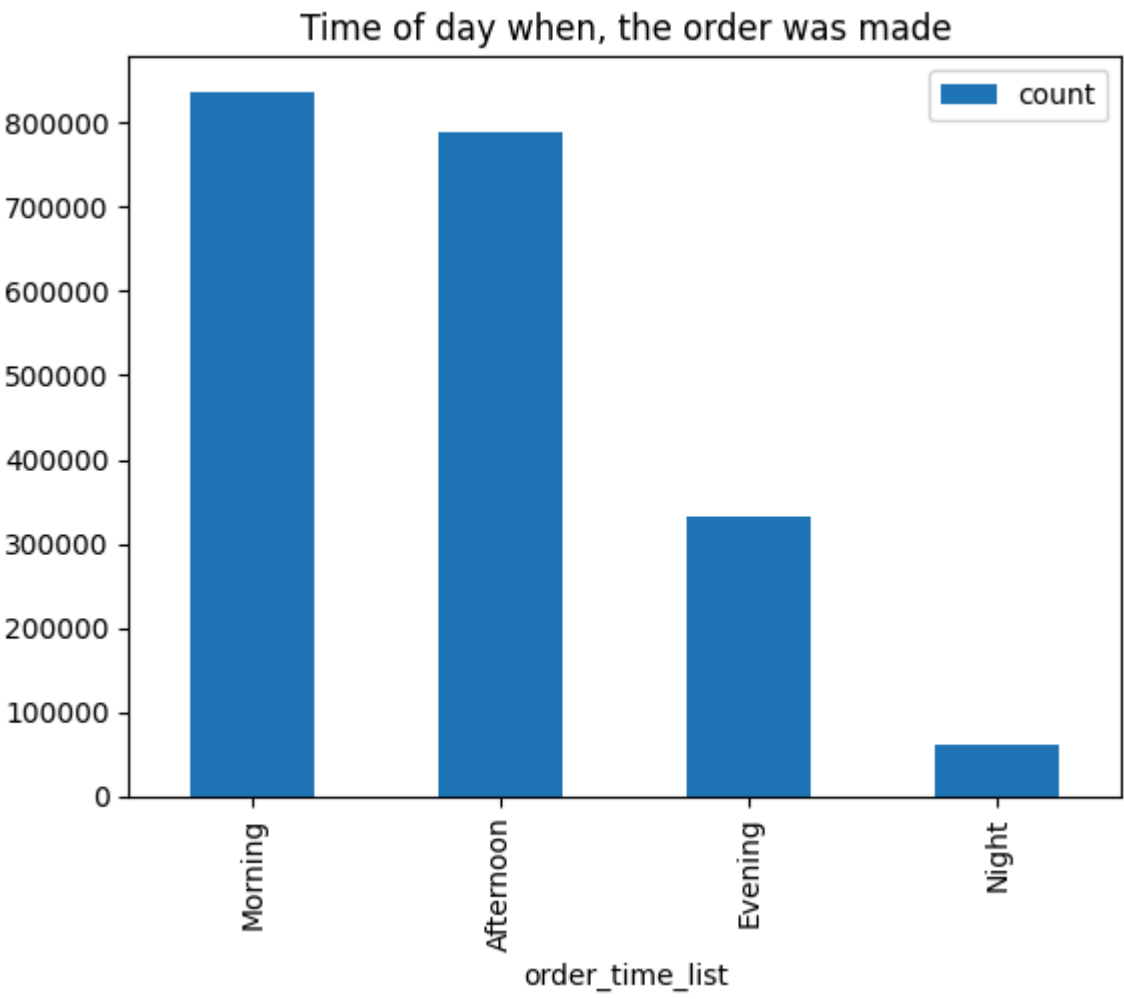
	count
order_time_list	
Morning	836546
Afternoon	788994
Evening	332661
Night	61300

In [31]:

```
result2.plot(
    kind = 'bar',
    title = 'Time of day when, the order was made'
)
```

Out[31]:

```
<Axes: title={'center': 'Time of day when, the order was made'}, xlabel='order_time_list'>
```



14. The time when the order was made by day and time of day

In [32]:

```
df.pivot_table(
    index = 'order_dow',
    columns = 'order_time_list',
    values = 'user_id',
    aggfunc = ['count']
)
```

Out[32]:

	count			
order_time_list	Afternoon	Evening	Morning	Night
order_dow				
0	162933	60195	157546	11157
1	130205	51825	158356	8850
2	100223	45497	108266	7926
3	94072	41737	95722	7199
4	89207	45359	92301	8017
5	101113	42311	110126	8607
6	111241	45737	114229	9544

In [33]:

```
result3 = df.pivot_table(
    index = 'order_dow',
    columns = 'order_time_list',
    values = 'user_id',
    aggfunc = ['count']
)
```

In [34]:

```
result3
```

Out[34]:

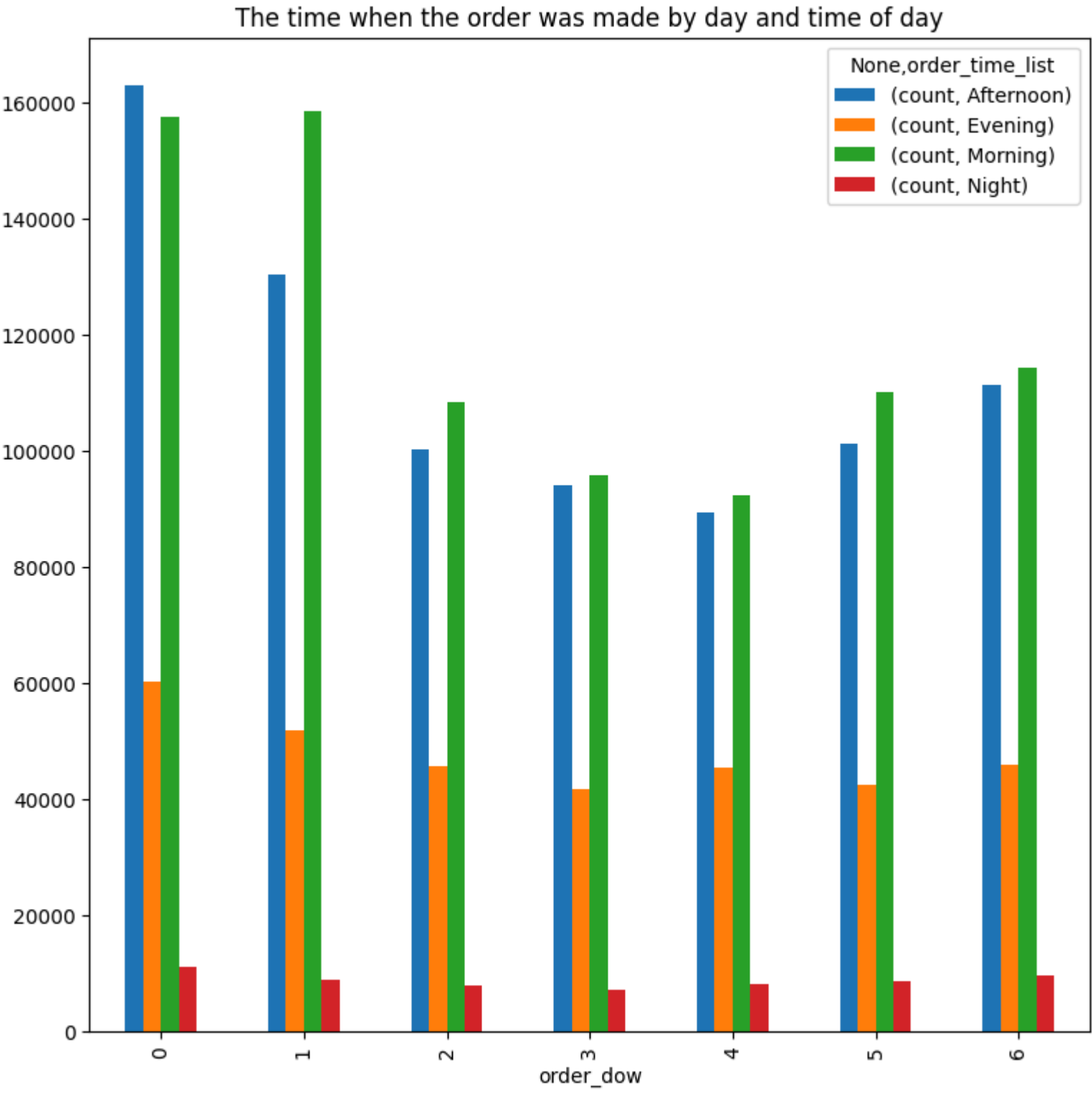
	count			
order_time_list	Afternoon	Evening	Morning	Night
order_dow				
0	162933	60195	157546	11157
1	130205	51825	158356	8850
2	100223	45497	108266	7926
3	94072	41737	95722	7199
4	89207	45359	92301	8017
5	101113	42311	110126	8607
6	111241	45737	114229	9544

In [35]:

```
result3.plot(
    kind = 'bar',
    title = 'The time when the order was made by day and time of day',
    figsize = (9,9)
)
```

Out[35]:

```
<Axes: title={'center': 'The time when the order was made by day and time of day'}, xlabel='order_dow'>
```

15. The number of orders that users have made

```
In [36]: df.groupby('order_number')['user_id'].agg(['count'])
```

Out[36]:

count	
order_number	
1	124342
2	122397
3	120627
4	118573
5	105482
...	...
96	1051
97	834
98	964
99	716
100	493

100 rows × 1 columns

```
In [37]: def order_number_group(x):
        if x <= 20 :
            x= '1-20 orders'
        elif x >20 and x <=40:
            x= '21-40 orders'
        elif x >40 and x <=60:
            x= '41-60 orders'
        elif x >60 and x <=80:
            x= '61-80 orders'
        else:
            x= '81-100 orders'
        return x
```

```
In [38]: df['order_number_group']=df['order_number'].apply(order_number_group)
```

```
In [39]: df
```

Out[39]:

	order_id	user_id	order_number	order_dow	order_hour_of_day	days_since_prior_order	product_id	add_to_cart_order	reordered	department_id	department	product_name	order_time_list	order_number_group	
	0	2425083	49125	1	2	18	0	17	1	0	13	pantry	baking ingredients	Evening	1-20 orders
	1	2425083	49125	1	2	18	0	91	2	0	16	dairy eggs	soy lactosefree	Evening	1-20 orders
	2	2425083	49125	1	2	18	0	36	3	0	16	dairy eggs	butter	Evening	1-20 orders
	3	2425083	49125	1	2	18	0	83	4	0	4	produce	fresh vegetables	Evening	1-20 orders
	4	2425083	49125	1	2	18	0	83	5	0	4	produce	fresh vegetables	Evening	1-20 orders
	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	2019496	3390742	199430	16	3	18	5	83	8	0	4	produce	fresh vegetables	Evening	1-20 orders
	2019497	458285	128787	42	2	19	3	115	1	1	7	beverages	water seltzer sparkling water	Evening	41-60 orders
	2019498	458285	128787	42	2	19	3	32	2	1	4	produce	packaged produce	Evening	41-60 orders
	2019499	458285	128787	42	2	19	3	32	3	1	4	produce	packaged produce	Evening	41-60 orders
	2019500	458285	128787	42	2	19	3	123	4	1	4	produce	packaged vegetables fruits	Evening	41-60 orders

2019501 rows × 14 columns

```
In [40]: result4 = df.groupby('order_number_group')['user_id'].agg(['count']).sort_values(by = 'count', ascending = False)
```

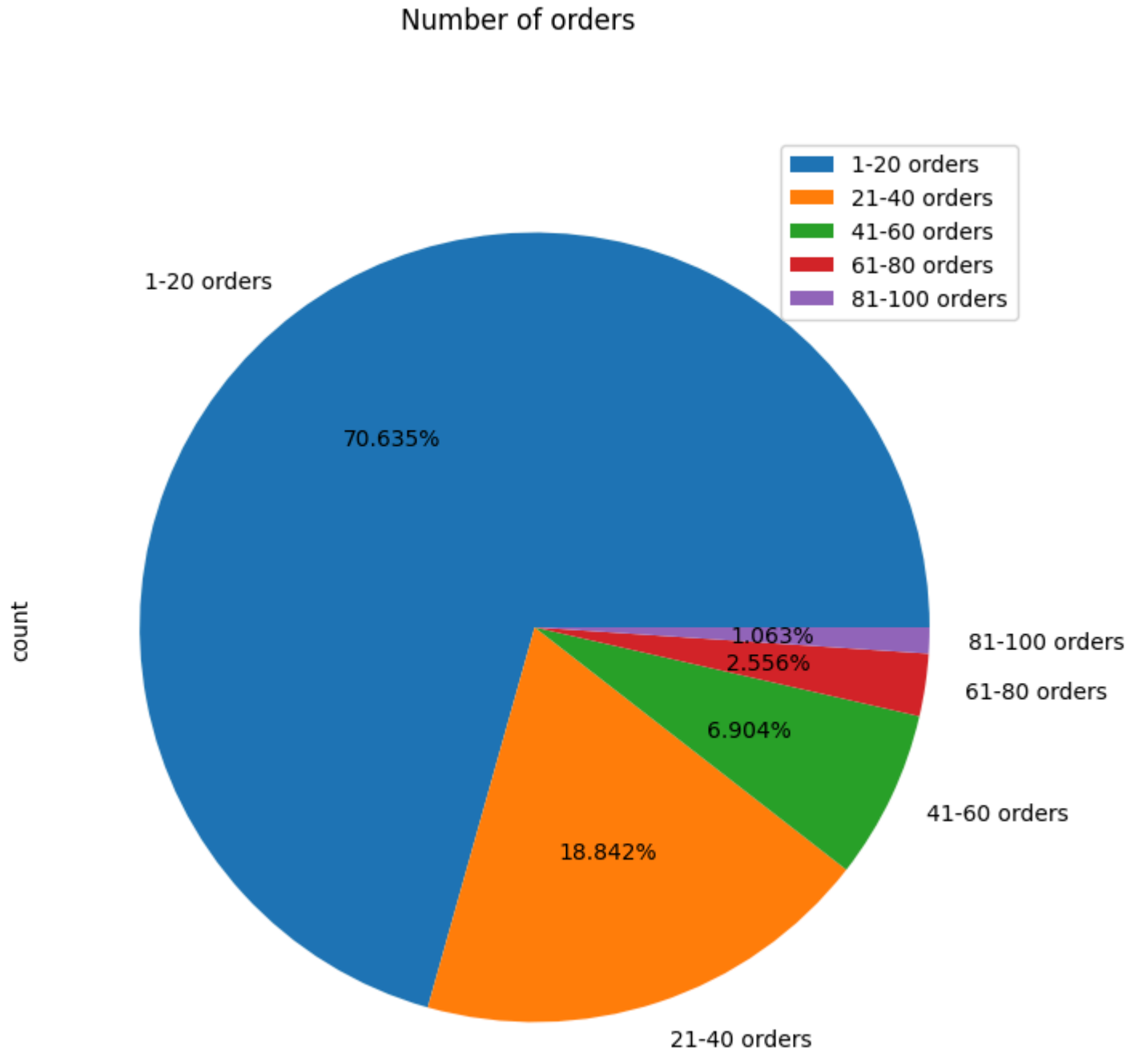
```
In [41]: result4
```

Out[41]:

count	
order_number_group	
1-20 orders	1426484
21-40 orders	380514
41-60 orders	139421
61-80 orders	51612
81-100 orders	21470

```
In [42]: result4.plot(
        kind = 'pie',
        autopct = '%1.3f%%',
        subplots = True,
        title = 'Number of orders',
        figsize = (8, 8)
    )
```

Out[42]: array([<Axes: ylabel='count'>], dtype=object)



16. How many days have passed since the pre-order

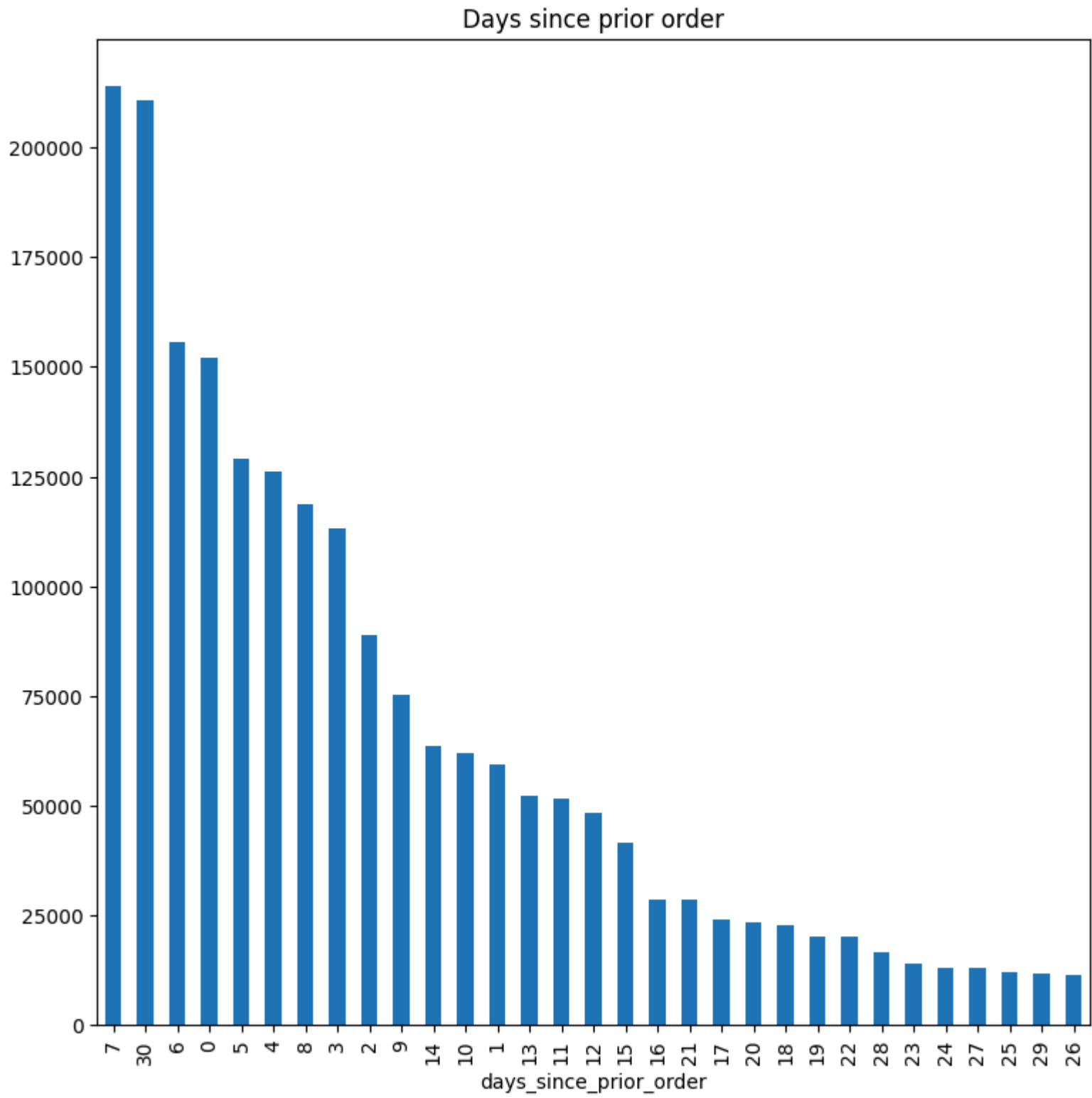
```
In [43]: result4 = df.groupby('days_since_prior_order')['user_id'].agg('count').sort_values(ascending = False)
```

```
In [44]: result4
```

```
Out[44]: days_since_prior_order
7      214126
30     210814
6      155685
0      152015
5      129089
4      126250
8      118722
3      113263
2       88737
9       75120
14      63414
10      62073
1       59200
13      52204
11      51476
12      48171
15      41531
16      28632
21      28359
17      23921
20      23234
18      22777
19      20140
22      19950
28      16459
23      13779
24      12833
27      12804
25      11832
29      11590
26       11301
Name: user_id, dtype: int64
```

```
In [45]: result4.plot(
        kind = 'bar',
        title = 'Days since prior order',
        figsize = (9,9)
    )
```

Out[45]: <Axes: title={'center': 'Days since prior order'}, xlabel='days_since_prior_order'>



17. Top 15 most popular products from users

```
In [46]: result5 = df.groupby('product_id')['user_id'].agg(['count']).sort_values(by = 'count',ascending = False).head(15)
```

```
In [47]: result5
```

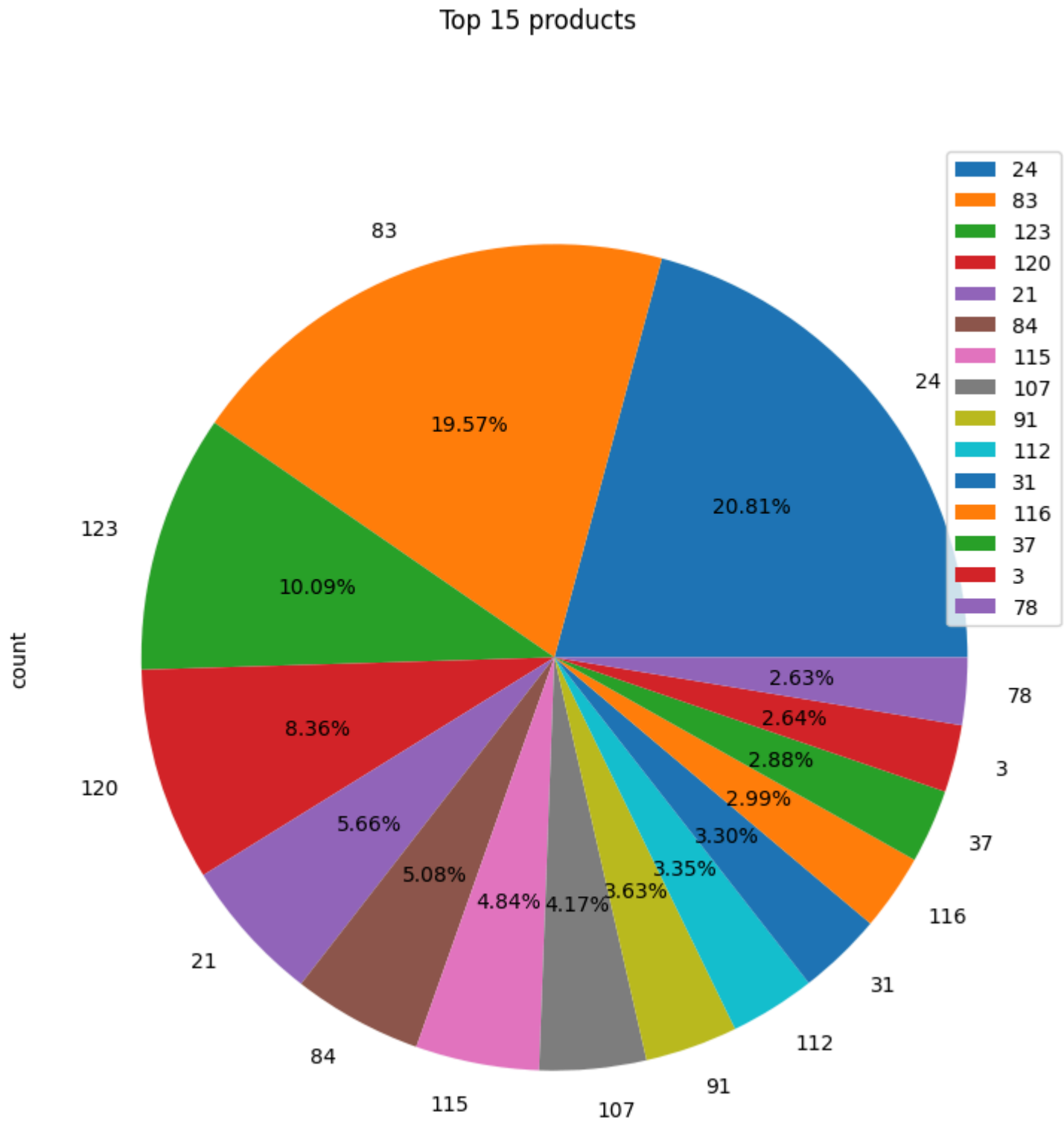
```
Out[47]: count
product_id
24      226039
83      212611
123     109596
120     90751
21      61502
84      55150
115     52564
107     45306
91      39389
112     36381
31      35893
116     32432
37      31323
3       28639
78      28574
```

```
In [48]: result5.plot(
        kind = 'pie',
```



```
autopct = '%1.2f%%',
subplots = True,
title = 'Top 15 products',
figsize = (9,9)
)
```

Out[48]: array([[<Axes: ylabel='count'>]], dtype=object)



In [49]: result6 = df.groupby('product_name')['user_id'].agg(['count']).sort_values(by = 'count',ascending = False).head(15)

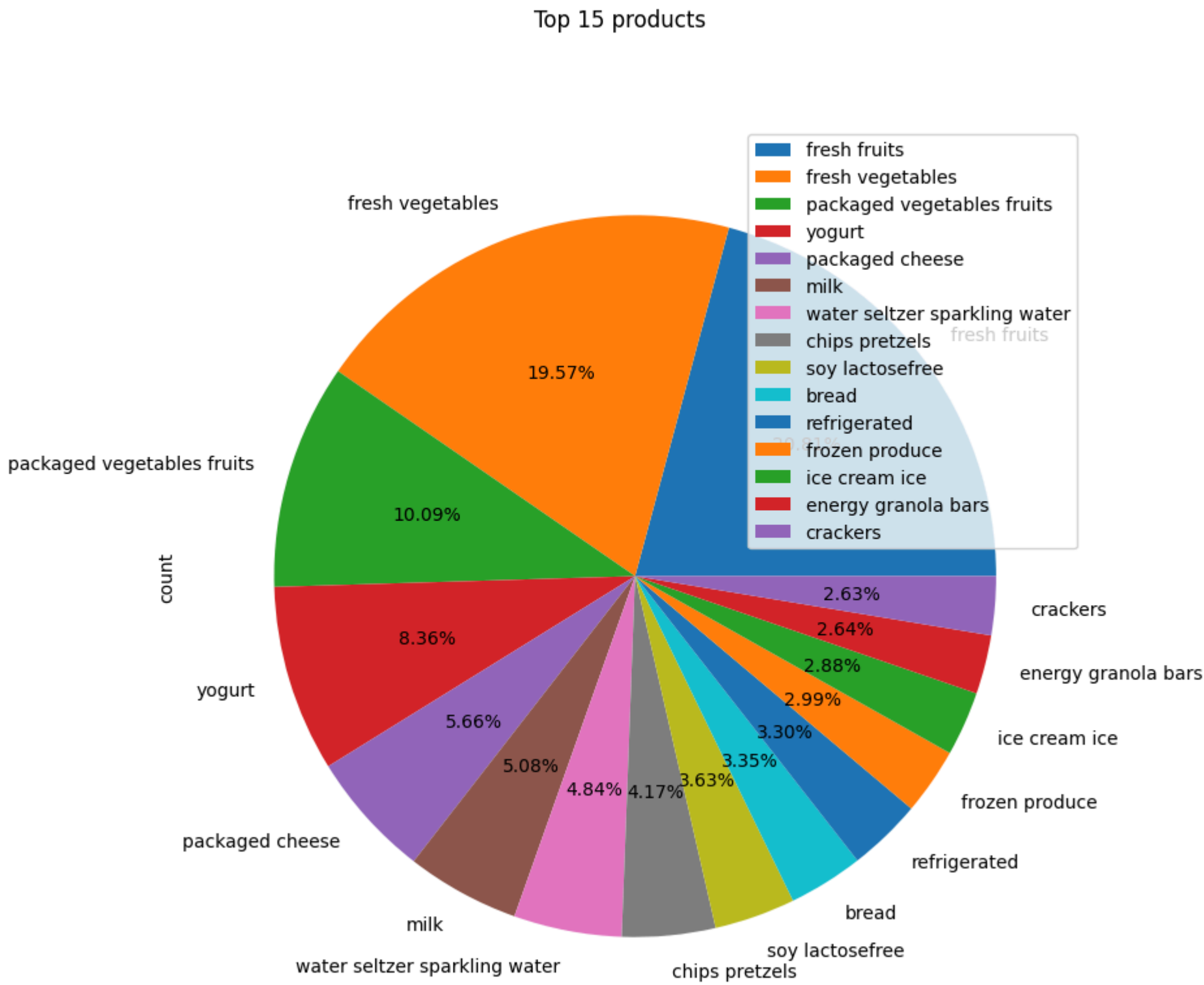
In [50]: result6

Out[50]:

	count
product_name	
fresh fruits	226039
fresh vegetables	212611
packaged vegetables fruits	109596
yogurt	90751
packaged cheese	61502
milk	55150
water seltzer sparkling water	52564
chips pretzels	45306
soy lactosefree	39389
bread	36381
refrigerated	35893
frozen produce	32432
ice cream ice	31323
energy granola bars	28639
crackers	28574

In [51]: result6.plot(kind='pie',autopct='%1.2f%%',subplots=True,title='Top 15 products',figsize=(9,9))

Out[51]: array([[<Axes: ylabel='count'>]], dtype=object)



18. How many products users add to the cart

In [52]: atco_u = df.groupby('add_to_cart_order', as_index = False).agg({'user_id': 'count'}).sort_values(by='user_id',ascending=False)

In [53]: atco_u

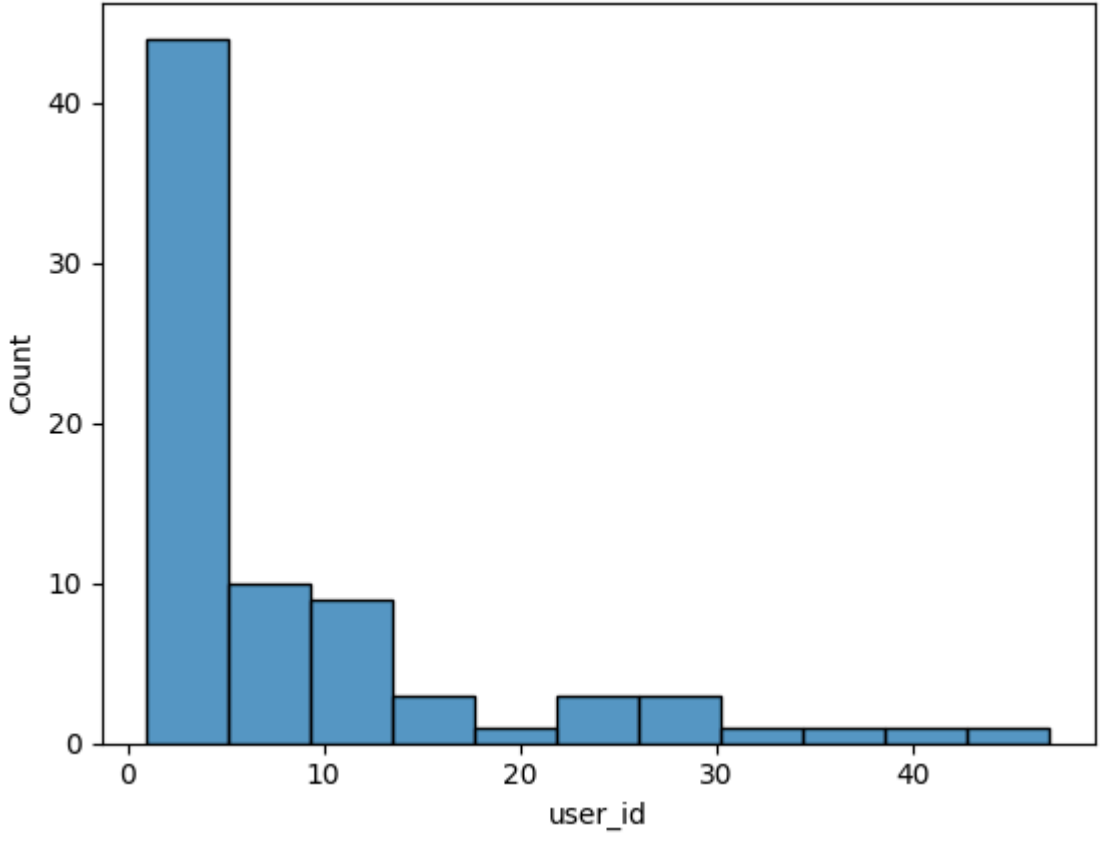
Out[53]:

add_to_cart_order	user_id
0	1 200000
1	2 190134
2	3 178480
3	4 165743
4	5 151983
...	...
112	113 1
111	112 1
110	111 1
109	110 1
136	137 1

137 rows × 2 columns

In [54]: sns.histplot(atco_u.query('user_id<50').user_id, kde = False)

Out[54]: <Axes: xlabel='user_id', ylabel='Count'>



19. How many products were added to the cart

```
In [55]: on_atco = df.groupby('order_number',as_index=False).agg({'add_to_cart_order': 'count'}).sort_values(by='add_to_cart_order', ascending = False)
```

```
In [56]: on_atco
```

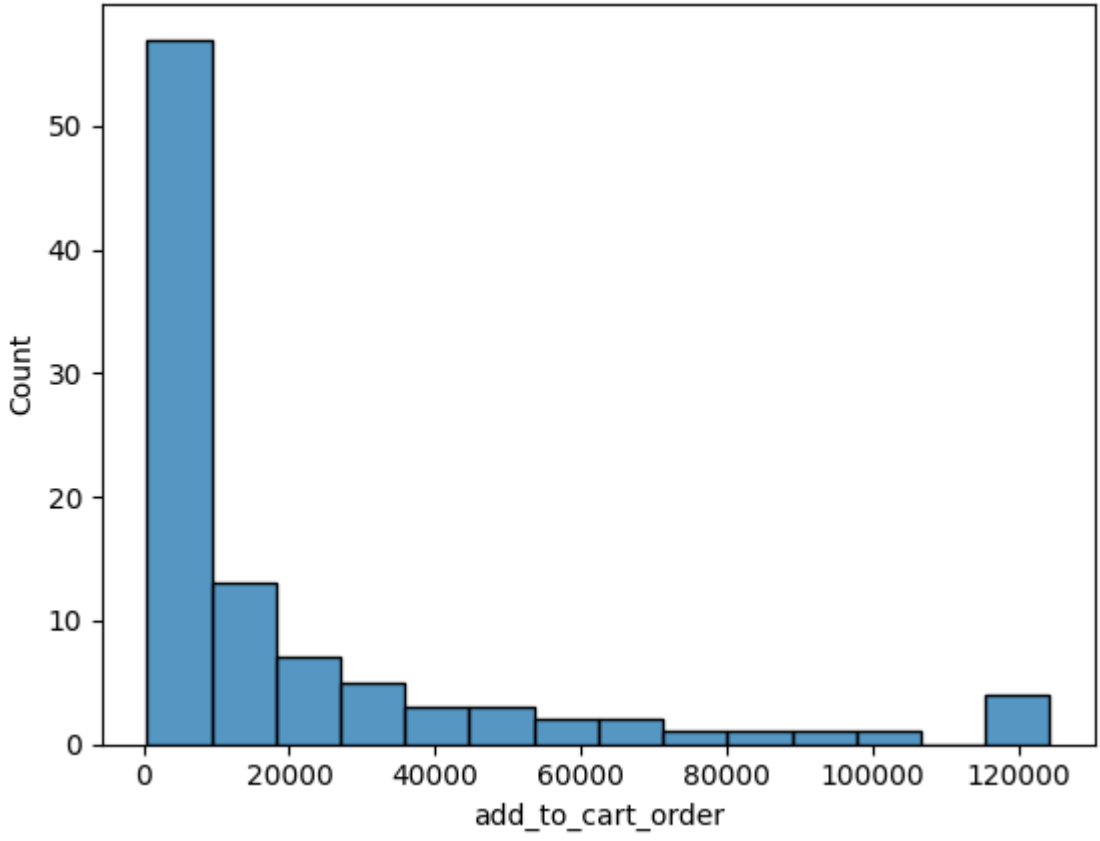
Out[56]:

order_number	add_to_cart_order
0	1
1	2
2	3
3	4
4	5
...	...
96	97
90	91
94	95
98	99
99	100

100 rows × 2 columns

```
In [57]: sns.histplot(on_atco.add_to_cart_order, kde=False)
```

```
Out[57]: <Axes: xlabel='add_to_cart_order', ylabel='Count'>
```

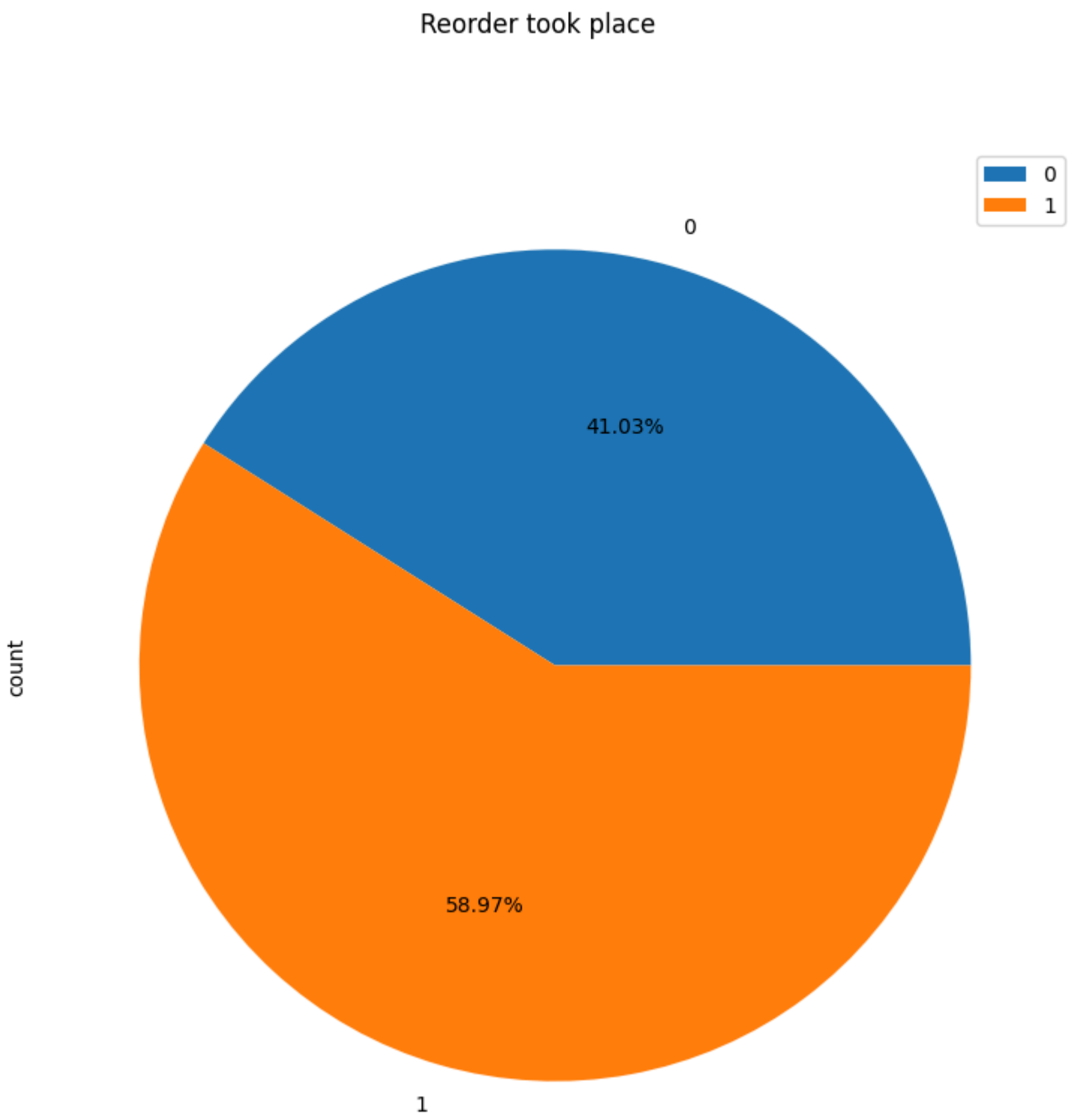


Out[63]:

count	
reordered	
0	828515
1	1190986

In [64]: result8.plot(kind='pie', autopct='%1.2f%', subplots=True, title='Reorder took place', figsize=(9,9))

Out[64]: array([[<Axes: ylabel='count'>]], dtype=object)



21. Top 10 reorders

In [65]: result9 = df.groupby('product_id')['reordered'].agg(['count']).sort_values(by = 'count', ascending = False).head(10)

In [66]: result9

Out[66]:

count	
product_id	
24	226039
83	212611
123	109596
120	90751
21	61502
84	55150
115	52564
107	45306
91	39389
112	36381

In [67]: result9.plot(kind = 'pie', autopct = '%1.3f%', subplots = True, title = 'Top 10 reordered', figsize = (9, 9))

Out[67]: array([[<Axes: ylabel='count'>]], dtype=object)

